College Research Affiliate Program Project

Harshendu V Kurup

Saintgits College of Engineering

Electronics and Computer

20/03/2026

INTRODUCTION

This project is an IoT-based Water Disaggregation Dashboard built for Saintgits College of Engineering. It collects real-time sensor data (water level and temperature) from IoT nodes, stores it in a cloud PostgreSQL database (Aiven), and uses Machine Learning models to predict water activity types such as shower, faucet, toilet, and dishwasher usage. The system features a React-based frontend dashboard with college branding, a FastAPI backend, and is fully deployed on the cloud using Render (backend) and Vercel (frontend).

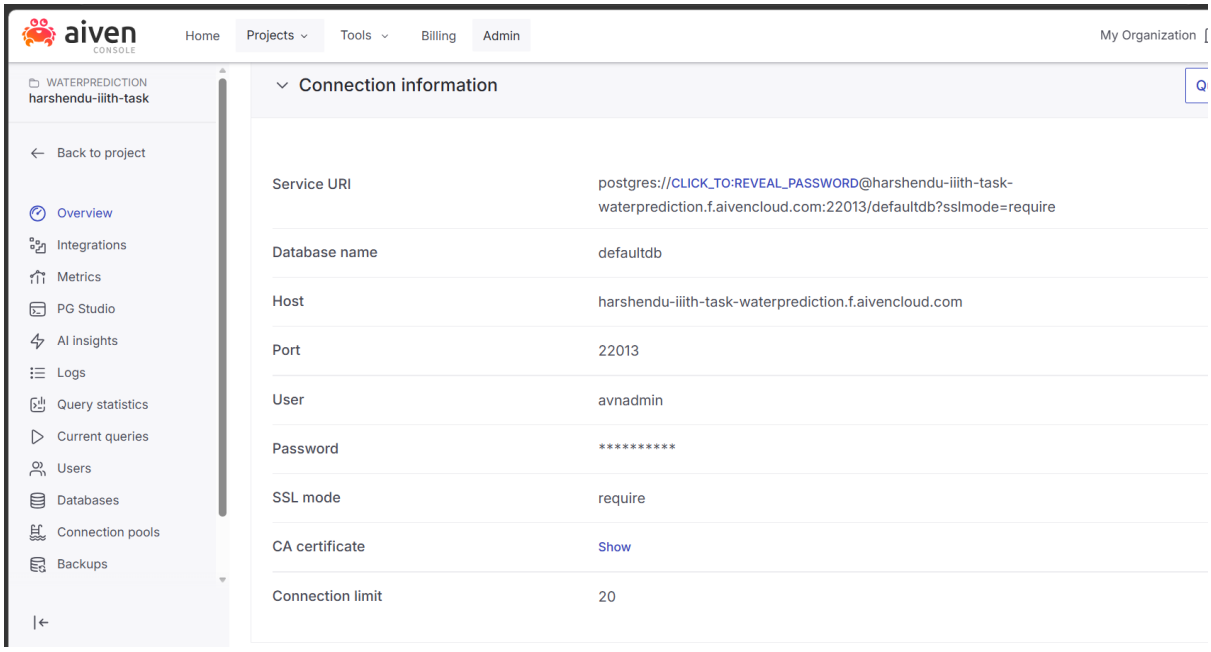
The goal is to demonstrate an end-to-end pipeline integrating:

- Data generation (simulated IoT)
- Machine Learning prediction
- Backend APIs
- Cloud database (PostgreSQL on Aiven)
- Frontend dashboard (React)
- Deployment (Render + Vercel)

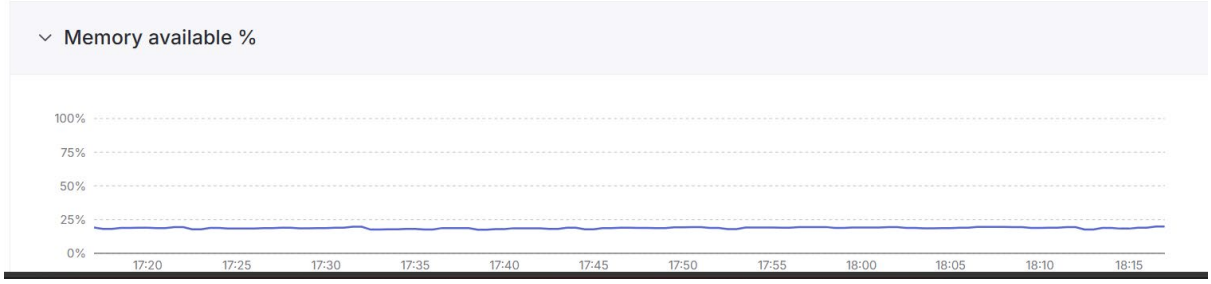
3. Backend Development

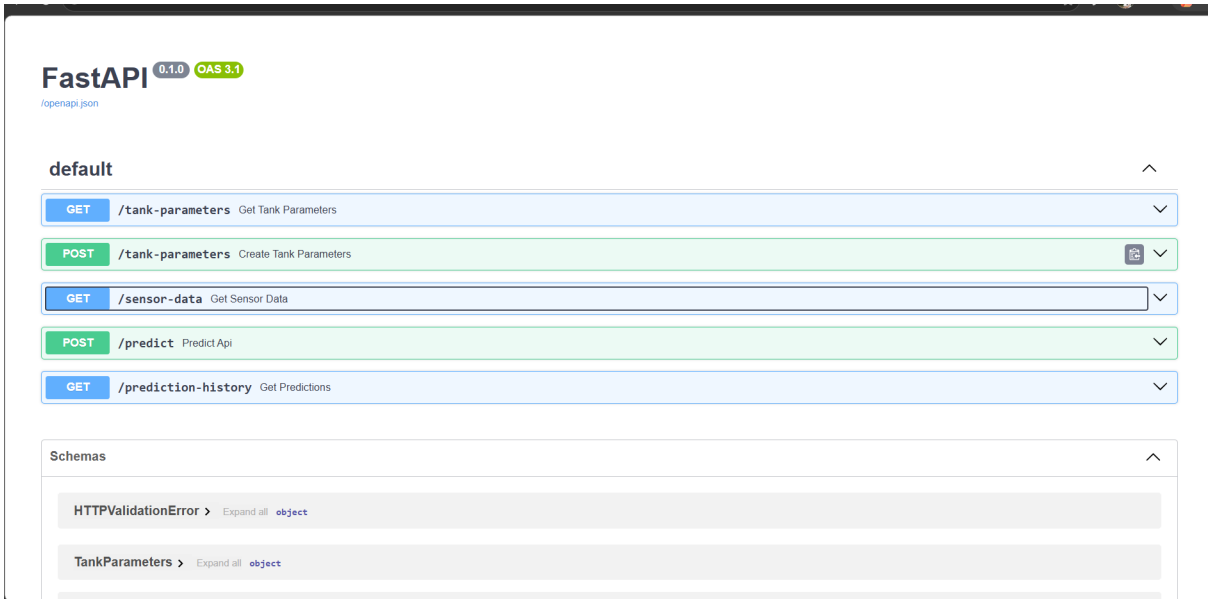
Technologies Used:

- FastAPI
- PostgreSQL (Aiven)
- Psycopg2
- Uvicorn


aiven
CONSOLE
Home
Projects
Tools
Billing
Admin
My Organization
WATERPREDICTION
harshendu-iiith-task
Back to project
Overview
Integrations
Metrics
PG Studio
AI insights
Logs
Query statistics
Current queries
Users
Databases
Connection pools
Backups
Connection information
Service URI
postgres://CLICK_TO:REVEAL_PASSWORD@harshendu-iiith-task-waterprediction.f.aivencloud.com:22013/defaultdb?sslmode=require
Database name
defaultdb
Host
harshendu-iiith-task-waterprediction.f.aivencloud.com
Port
22013
User
avnadmin
Password
****
SSL mode
require
CA certificate
Show
Connection limit
20

Load average (5 min)
0.26
0.195
0.13
0.065
0
17:20
17:25
17:30
17:35
17:40
17:45
17:50
17:55
18:00
18:05
18:10
18:15


Memory available %
100%
75%
50%
25%
0%
17:20
17:25
17:30
17:35
17:40
17:45
17:50
17:55
18:00
18:05
18:10
18:15



Features Implemented:

- API for prediction (/predict)
- API for prediction history (/prediction-history)
- Database integration with Aiven
- Background task for continuous data generation
- Automatic table creation

Key Learning:

- Handling environment variables

- Connecting cloud database securely
- Writing REST APIs
- Debugging server-side errors

---

4. Machine Learning Model

Work Done:

- Used dataset provided in notebook
- Trained multiple models:
  - KNN
  - Random Forest
- Compared using:
  - Accuracy
  - F1 Score

Result:

- KNN ≈ 99.48% accuracy
- Random Forest ≈ 99.38% accuracy

Final Step:

- Selected best model
- Saved using joblib
- Integrated into backend

---

5. Database Integration (Aiven PostgreSQL)

Tasks Completed:

- Created cloud PostgreSQL instance
- Connected backend using environment variables
- Created tables:
  - sensor_data
  - predictions

Challenges Faced:

- Authentication errors
- Column type mismatch (FLOAT vs TEXT)
- Schema modification issues

Solution:

- Used backend-based fixes instead of manual DB edits
- Implemented safe schema migration

---

6. Frontend Development

Technologies:

- React.js
- React Router

Features Added:

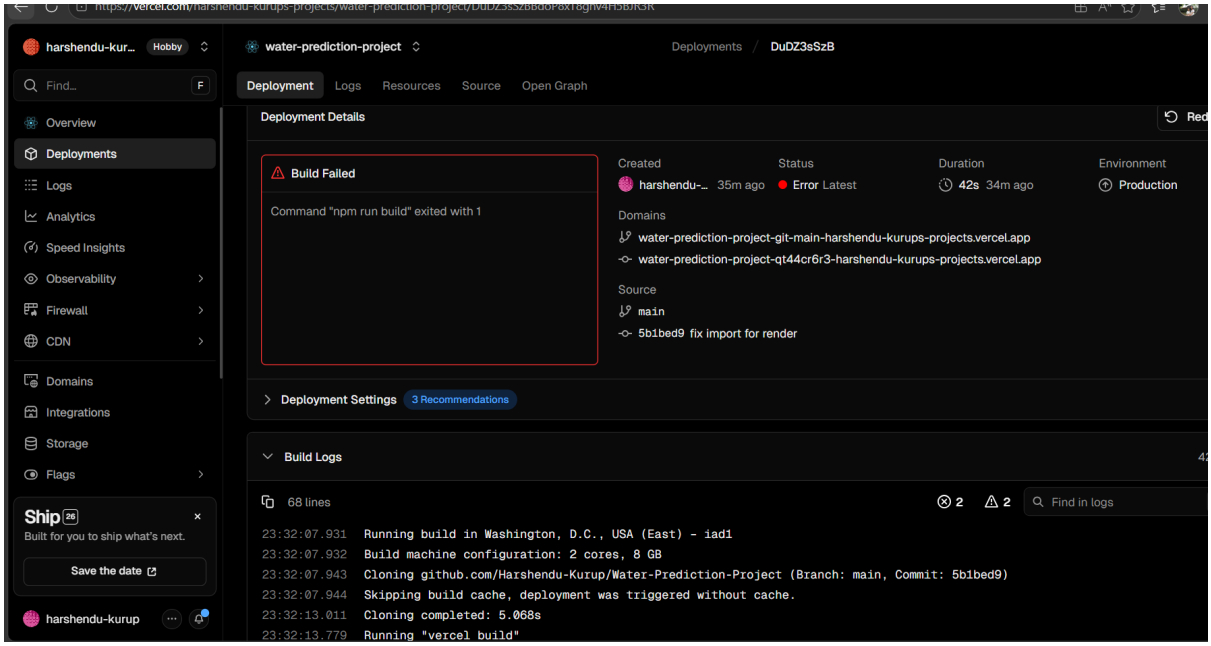
- Dashboard (existing)
- Prediction Page (new)
- History Page (new)

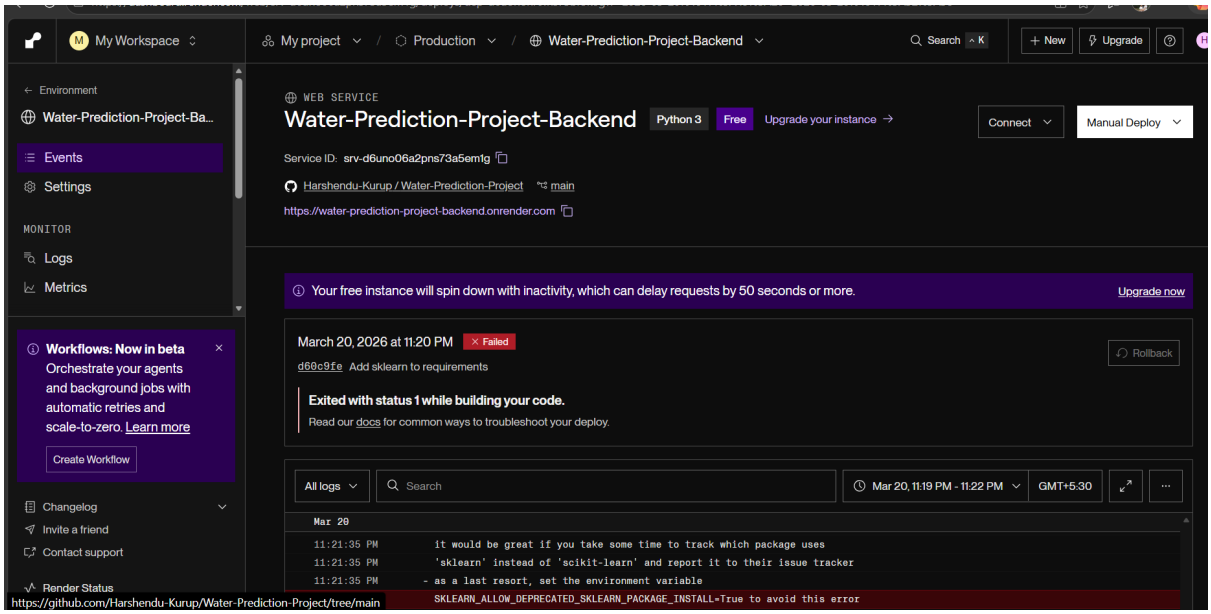
Prediction Page:

- Calls backend /predict API
- Displays real-time prediction

History Page:

- Fetches past predictions
- Displays in table format





---

7. Deployment

Backend:

- Platform: Render
- Configured:
  - Build command
  - Start command

    - Dependencies

Frontend:

- Platform: Vercel
- Connected GitHub repository
- Set root directory as frontend

---

8. Challenges Faced

1. Database Errors

- Password authentication failure
- Column type mismatch
- Difficulty viewing data in Aiven UI

👉 Solved by:

- Using backend APIs for verification
- Avoiding manual DB edits

---

2. Model Integration Confusion

- Understanding input format
- Difference between frontend input vs DB data

👉 Learned:

- Data pipeline flow is crucial
- Model depends on input distribution

---

3. Backend Debugging

- Module import errors
- Path issues during deployment
- Missing dependencies

👉 Solved by:

- Fixing import paths

- Updating requirements.txt

---

4. Frontend Issues

- Routing errors
- ESLint import errors
- Duplicate imports

👉 Solved by:

- Cleaning App.js structure
- Proper routing setup

---

5. Deployment Issues

- Render crashes
- Missing packages
- Incorrect file paths

👉 Learned:

- Deployment environment differs from local setup
- Proper configuration is critical

---

9. Current Status

Completed:

✓ Backend APIs working
✓ ML model integrated
✓ Database connected
✓ Prediction working
✓ Frontend UI created
✓ Prediction & history pages added

In Progress:

⏳ Final frontend deployment fix
⏳ Backend deployment stabilization

---

10. Key Learnings

- End-to-end system design
- Backend + frontend integration
- Debugging real-world errors
- Cloud deployment challenges
- Importance of data flow in ML systems

---

11. Conclusion

This project successfully demonstrates a full-stack AI system integrating machine learning, cloud databases, backend APIs, and frontend visualization.

Despite multiple debugging challenges, the system is functional and showcases real-world engineering skills including:

- Problem solving
- System integration
- Deployment handling

---

12. Future Improvements

- Improve UI/UX design
- Add real sensor integration
- Enhance model with more features
- Add real-time charts
- Optimize deployment pipeline